

Stage 7 — Auth, Multi-Tenancy, and Rate Limiting

Goal: Add authentication, tenant isolation, and basic rate limiting on top of your Stage 1–6 architecture using JWT/API keys + Postgres (with optional Row-Level Security) + per-tenant rate limits.

This turns your Creator Orchestrator into a real SaaS-style, multi-tenant service while keeping everything aligned with enterprise security and least-privilege standards.
[web:220][web:223][web:225][web:231][web:232]

1. Where This Fits (and What's Left)

You now have:

1. Stage 1 — Core agent loop + tools.
2. Stage 2 — Orchestrator + multiple agents.
3. Stage 3 — Memory: Postgres + pgvector.
4. Stage 4 — Guardrails & RBAC.
5. Stage 5 — Evals & observability.
6. Stage 6 — Production repo layout + HTTP API (FastAPI, Docker).
7. Stage 7 — Auth, multi-tenant model, and rate limiting (this document).

After Stage 7, the only major **optional** stage left is billing/quotas/feature flags (Stage 8). Architecturally, Stage 7 gives you a complete SaaS-ready spine.

2. Design Overview — Identities, Tenants, and Limits

Modern API security guidance converges on a few patterns:[web:220][web:221][web:223]
[web:229]

- Use **JWT** (Bearer tokens) for user authentication against HTTP APIs.
- Optionally combine this with **API keys** for machine-to-machine (M2M) access.
- Model **tenants** explicitly and ensure isolation at the data layer (Row-Level Security in Postgres).[web:225][web:230]
- Implement **rate limits** per user/tenant/client to mitigate abuse and control LLM costs.
[web:226][web:231][web:233]
- Apply **least privilege + zero trust**: every request is authenticated, authorized, and rate-limited; agents/tools never rely on "trust the network".[web:227][web:232][web:233]

Stage 7 adds three things to your stack:

1. **Auth layer** — JWT login endpoint and API key support.

2. **Multi-tenant data model** — tenants, users, user_tenants, api_keys, and tenant_id columns on core tables, with optional RLS.
3. **Rate limiting layer** — per-tenant and per-user request counters in Postgres (with an interface that can later swap to Redis or an API gateway).

3. Database Schema: Users, Tenants, API Keys, Limits

3.1 Migration: 004_auth_multitenant_ratelimit.sql

```
-- Tenants (organizations or accounts)
CREATE TABLE IF NOT EXISTS tenants (
    id          UUID PRIMARY KEY,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    name        TEXT NOT NULL,
    metadata    JSONB DEFAULT '{}'::jsonb
);

-- Users
CREATE TABLE IF NOT EXISTS users (
    id          UUID PRIMARY KEY,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    email       TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    is_active   BOOLEAN NOT NULL DEFAULT true,
    metadata    JSONB DEFAULT '{}'::jsonb
);

-- User ↔ Tenant membership with role
CREATE TABLE IF NOT EXISTS user_tenants (
    user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    role        TEXT NOT NULL DEFAULT 'member', -- 'owner', 'admin', 'member'
    PRIMARY KEY (user_id, tenant_id)
);

-- API keys for server-to-server access
CREATE TABLE IF NOT EXISTS api_keys (
    id          UUID PRIMARY KEY,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    user_id     UUID REFERENCES users(id) ON DELETE SET NULL,
    label       TEXT NOT NULL,
    key_hash    TEXT NOT NULL,          -- hashed API key (never store raw)
    is_revoked  BOOLEAN NOT NULL DEFAULT false,
    last_used_at TIMESTAMPTZ,
    metadata    JSONB DEFAULT '{}'::jsonb
);

-- Extend existing tables with tenant / user context (if not already)
ALTER TABLE sessions ADD COLUMN IF NOT EXISTS tenant_id UUID;
ALTER TABLE runs     ADD COLUMN IF NOT EXISTS tenant_id UUID;
ALTER TABLE messages ADD COLUMN IF NOT EXISTS user_id   UUID;
```

```

ALTER TABLE memories    ADD COLUMN IF NOT EXISTS tenant_id UUID;
ALTER TABLE guardrail_events ADD COLUMN IF NOT EXISTS tenant_id UUID;
ALTER TABLE eval_results    ADD COLUMN IF NOT EXISTS tenant_id UUID;

-- Basic rate limiting table: counters per tenant/user/period
CREATE TABLE IF NOT EXISTS api_usage_counters (
    id            BIGSERIAL PRIMARY KEY,
    tenant_id     UUID NOT NULL,
    user_id       UUID,
    period_start  TIMESTAMPTZ NOT NULL, -- floor to minute/hour
    period        TEXT NOT NULL,        -- 'minute', 'hour', 'day'
    requests      INTEGER NOT NULL DEFAULT 0,
    tokens        BIGINT NOT NULL DEFAULT 0
);

CREATE INDEX IF NOT EXISTS idx_api_usage_tenant_period
ON api_usage_counters (tenant_id, user_id, period, period_start);

```

Note: In a large deployment you might use Redis or an API gateway for rate limiting, but Postgres-backed counters are acceptable at early stages and are fully consistent with an enterprise stack when traffic is moderate.[web:226][web:231]

4. Optional: Row-Level Security for Strong Tenant Isolation

Postgres Row-Level Security (RLS) lets you enforce tenant isolation directly in the DB, not just in app code.[web:225][web:230]

High-level steps:[web:225][web:230]

1. Create an application DB role (e.g., `app_user`) without `BYPASSRLS`.
2. Add a **session variable** `app.current_tenant` per connection.
3. Enable RLS and write policies that compare table `tenant_id` with `current_setting('app.current_tenant')`.

Example for runs:

```

ALTER TABLE runs ENABLE ROW LEVEL SECURITY;

CREATE POLICY runs_tenant_isolation ON runs
    USING (tenant_id::text = current_setting('app.current_tenant', true));

```

Then, in your DB helper (`memory/db.py`), you set this variable at the start of each request (see section 6).

RLS reduces the risk of cross-tenant data leaks from coding mistakes by enforcing the tenant boundary at the DB layer itself.[web:225][web:230]

5. Auth Model: JWT + API Keys

5.1 JWT for user authentication

JWT (JSON Web Token) is a widely used stateless auth mechanism for APIs:[web:220][web:223][web:224][web:229]

- The user logs in with email/password.
- Server verifies credentials and issues a signed JWT containing `user_id` and `tenant_id` (and roles).
- Client sends this token in the `Authorization: Bearer <token>` header on subsequent requests.[web:223][web:224]
- Server validates the signature and expiry on every request; no server-side session storage is needed.

Best practices:

- Short-lived access tokens (e.g., 5–30 minutes).[web:224]
- Strong signing key or asymmetric key pair.
- HTTPS enforced for all traffic.[web:223][web:224]
- Do not put sensitive data (passwords, secrets) in JWT payload.

Example: `app/auth.py`

```
# app/auth.py
from datetime import datetime, timedelta, timezone
from typing import Optional
from uuid import uuid4

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError
from passlib.context import CryptContext

from app.config import settings
from memory.db import get_cursor

SECRET_KEY = "change-me" # load from env in production
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/token")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

```

def get_user_by_email(email: str):
    with get_cursor() as cur:
        cur.execute("SELECT id, password_hash FROM users WHERE email = %s AND is_active = 1")
        row = cur.fetchone()
        if not row:
            return None
        return {"id": row[0], "password_hash": row[1]}

def get_primary_tenant_id(user_id):
    with get_cursor() as cur:
        cur.execute("SELECT tenant_id FROM user_tenants WHERE user_id = %s LIMIT 1",
                    (user_id,))
        row = cur.fetchone()
        return row[0] if row else None

def authenticate_user(email: str, password: str):
    user = get_user_by_email(email)
    if not user:
        return None
    if not verify_password(password, user["password_hash"]):
        return None
    return user

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

async def get_current_principal(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")
        tenant_id: str = payload.get("tenant_id")
        if user_id is None or tenant_id is None:
            raise JWTError()
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Could not validate credentials",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return {"user_id": user_id, "tenant_id": tenant_id}

```

Auth endpoints in app/main.py

```

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from app.auth import authenticate_user, create_access_token, get_primary_tenant_id

router = APIRouter(prefix="/auth", tags=["auth"])

```

```

@router.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Incorrect username or password")
    tenant_id = get_primary_tenant_id(user["id"])
    access_token = create_access_token(data={"sub": str(user["id"]), "tenant_id": str(tenant_id)})
    return {"access_token": access_token, "token_type": "bearer"}

```

Any protected endpoint then depends on `get_current_principal` to get `user_id` and `tenant_id`.

5.2 API keys for M2M access

API keys remain common for machine-to-machine scenarios, but should be scoped and managed carefully.[web:220][web:221][web:229][web:234]

Key practices:[web:221][web:224][web:229][web:234]

- Store **only a hash** of the key in DB.
- Provide friendly labels and the ability to revoke/regenerate.
- Pass keys in the `Authorization: Bearer <key>` header or `X-API-Key` header, not as query params.[web:224]
- Use keys for client identity; combine with tenant context and RBAC.

Example: `app/api_key_auth.py` (simplified):

```

from fastapi import Depends, HTTPException, status, Header
from typing import Optional
from hashlib import sha256

from memory.db import get_cursor

async def get_principal_from_api_key(authorization: Optional[str] = Header(None)):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Missing or invalid authorization header")

    raw_key = authorization.split(" ", 1)[1]
    key_hash = sha256(raw_key.encode("utf-8")).hexdigest()

    with get_cursor() as cur:
        cur.execute(
            """
            SELECT tenant_id, user_id
            FROM api_keys
            WHERE key_hash = %s AND is_revoked = false
            """,
            (key_hash,),
        )
        row = cur.fetchone()

    if not row:

```

```

        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid
tenant_id, user_id = row
return {"user_id": str(user_id) if user_id else None, "tenant_id": str(tenant_id)}

```

You can support **either JWT or API key** by trying JWT first and falling back to API key, or have separate endpoints.

6. Setting Tenant Context for DB (RLS-Ready)

To use `tenant_id` everywhere (and RLS if enabled), you need to set a per-request context in the DB connection.[\[web:225\]](#)

Modify `memory/db.py` to accept optional `tenant_id` and set `app.current_tenant`:

```

# memory/db.py
import psycopg
from contextlib import contextmanager
from config import DATABASE_URL

@contextmanager
def get_conn(tenant_id: str | None = None):
    conn = psycopg.connect(DATABASE_URL)
    if tenant_id:
        with conn.cursor() as cur:
            cur.execute("SELECT set_config('app.current_tenant', %s, true)", (tenant_id,))
    try:
        yield conn
    finally:
        conn.close()

@contextmanager
def get_cursor(tenant_id: str | None = None):
    with get_conn(tenant_id) as conn:
        with conn.cursor() as cur:
            yield cur
        conn.commit()

```

Then, in API layer and orchestrator, always call `get_cursor(tenant_id=principal["tenant_id"])`. This ensures RLS policies (if enabled) and tenant scoping are consistently applied.[\[web:225\]](#)[\[web:230\]](#)

7. Integrating Auth with the Orchestrator Endpoint

Update `/api/orchestrate` in `app/main.py` to require authentication and propagate tenant/user IDs.

```

from fastapi import Depends
from app.auth import get_current_principal

@app.post("/api/orchestrate", response_model=OrchestrateResponse)
async def orchestrate(payload: OrchestrateRequest,
                      principal = Depends(get_current_principal),
                      orchestrator = Depends(get_orchestrator)):
    tenant_id = principal["tenant_id"]
    user_id = principal["user_id"]

    session_id = short_term.create_session(user_id=user_id, topic=payload.topic)
    run_id = episodic.start_run(session_id, orchestrator="http-api", metadata={})

    with run_timer(run_id, session_id) as metrics:
        result = orchestrator.run(
            topic=payload.topic,
            user_id=user_id,
            session_id=session_id,
            run_id=run_id,
            brief=payload.brief,
        )

    episodic.finish_run(run_id, status="success")
    # ... map result to OrchestrateResponse as before ...

```

Now every call is associated with a **user + tenant**, and that context flows into memory, guardrails, and evals.

8. Rate Limiting per Tenant/User

Rate limiting is essential for LLM APIs to control costs and mitigate abuse.[web:226][web:231][web:233]

Targets:

- **Per-tenant limits** (e.g., 60 requests/minute, 10k tokens/minute).
- **Optional per-user limits** to catch abusive users inside a tenant.

For Stage 7, implement a simple Postgres-backed rate limiter with a sliding window per tenant.[web:226][web:231]

8.1 app/rate_limit.py

```

# app/rate_limit.py
from datetime import datetime, timezone, timedelta
from fastapi import HTTPException, status

from memory.db import get_cursor

MAX_REQUESTS_PER_MINUTE = 60

```



```

def floor_to_minute(dt: datetime) -> datetime:
    return dt.replace(second=0, microsecond=0)

def enforce_rate_limit(tenant_id: str, user_id: str | None) -> None:
    now = datetime.now(timezone.utc)
    period_start = floor_to_minute(now)

    with get_cursor(tenant_id=tenant_id) as cur:
        # Upsert a counter row for this minute
        cur.execute(
            """
            INSERT INTO api_usage_counters (tenant_id, user_id, period_start, period_end)
            VALUES (%s, %s, %s, 'minute')
            ON CONFLICT (tenant_id, user_id, period_start, period_end)
            DO UPDATE SET requests = api_usage_counters.requests + 1
            RETURNING requests
            """,
            (tenant_id, user_id, period_start),
        )
        requests = cur.fetchone()[0]

    if requests > MAX_REQUESTS_PER_MINUTE:
        # Optional: include retry-after header in real implementation
        raise HTTPException(
            status_code=status.HTTP_429_TOO_MANY_REQUESTS,
            detail="Rate limit exceeded. Please try again later.",
        )

```

You can later extend this to track token counts as well (using `RunMetrics`) and enforce token-per-minute or token-per-day limits.[web:231]

8.2 Calling the rate limiter in the API

In `/api/orchestrate` before invoking the orchestrator:

```

from app.rate_limit import enforce_rate_limit

@app.post("/api/orchestrate", response_model=OrchestrateResponse)
async def orchestrate(payload: OrchestrateRequest,
                      principal = Depends(get_current_principal),
                      orchestrator = Depends(get_orchestrator)):
    tenant_id = principal["tenant_id"]
    user_id = principal["user_id"]

    enforce_rate_limit(tenant_id, user_id)

    # ... rest remains the same ...

```

Best practice details:

- Return HTTP 429 with clear message and optional `Retry-After` header.[web:226][web:231]

- Log rate limit events (could be another `guardrail_event` or a dedicated table).
- Different limits per plan/tier (store in `tenants.metadata` or a `plans` table).

9. Interaction with Guardrails and Least-Privilege

Your Stage 4 guardrails + Stage 7 auth/rate-limit combine into a strong least-privilege model for AI agents:[web:227][web:232][web:233]

- **Auth & tenant context** ensure every request has a clear identity and tenant boundary.
- **RLS (optional)** enforces tenant isolation at the DB level.[web:225]
- **Tool RBAC** (Stage 4) restricts which tools a user/role can invoke.[web:191][web:197]
- **Guardrails** (Stage 4) filter inputs/outputs and block unsafe behavior.[web:188][web:189]
- **Rate limits** (Stage 7) constrain request volume and blast radius.[web:226][web:231][web:233]

This layered approach matches OWASP LLM Top-10 mitigation advice and zero-trust guidance for AI agents.[web:228][web:232][web:233]

10. Step-by-Step Checklist for Stage 7

1. **Apply migration 004** to add `users/tenants/api_keys` and `tenant_id` columns.
2. **Seed base data** using `scripts/seed.py`:
 - Create one tenant for yourself.
 - Create one user with a `bcrypt` `password_hash`.
 - Link them in `user_tenants`.
3. **Implement** `app/auth.py` and auth endpoints (`/auth/token`).
4. **Protect** `/api/orchestrate` using `get_current_principal` and pass `user_id`, `tenant_id` into orchestrator and memory.
5. **Update** `memory/db.py` to set `app.current_tenant` and (optionally) enable RLS policies.
6. **Add API key support** for M2M if needed.
7. **Implement** `enforce_rate_limit` and call it at the start of `/api/orchestrate`.
8. **Run end-to-end**:
 - `POST /auth/token` with your credentials → receive JWT.
 - `POST /api/orchestrate` with `Authorization: Bearer <token>` → see orchestrator output.
 - Exceed rate limit → receive 429 and see counters update.

When you can do all of this and explain how `user` → `tenant` → `run` → `guardrails` → `evals` are connected for a single request, Stage 7 is complete and your architecture is at a genuinely enterprise-ready level.

11. What's Next (Optional Stage 8)

Stage 8 (optional) would layer on:

- **Plans & billing** (Stripe/Paddle integration).
- **Per-plan rate/usage limits** tied into `api_usage_counters` and `tenants`.
- **Feature flags** (which models/tools are enabled per tier).

Those are business-side concerns. Architecturally, Stage 1–7 now form a coherent, production-grade AI agent platform.